

UNIVERSITE NUMERIQUE CHEIKH HAMIDOU KANE
(UNCHK)

Pole Sciences, Technologie et Numerique
Master 1 Big Data Analytics : Annee 2024/2025

TRAVAUX PRATIQUES

Cours : Programmation Parallele

Paralleliser un calcul de moyenne avec Numba

Etudiante	Ndeye Sokhna Nokho
Formation	Master 1 Big Data Analytics : UNCHK
Enseignant	Prof. Osias Noel TOSSOU (AIMS Senegal / UVS)
GitHub	https://github.com/Nokho11/TP_NUMBA_Programmation_Parallele

Resultats cles obtenus sur MacBook Pro M1 (8 threads)

Temps sequentiel T_1	0.0059 s
Temps parallele T_p	0.0011 s
Speedup mesure S	5.30
Proportion parallelisable P	92.72 %
Speedup max theorique ($N \rightarrow \infty$)	13.75

Contents

1	Introduction	3
2	Presentation du probleme	3
2.1	Enonce et coefficients	3
2.2	Formule de calcul	4
3	Generation des donnees	4
3.1	Approche	4
3.2	Code Python : Generation CSV	4
3.3	Resultat de l'execution	5
4	Version sequentielle	5
4.1	Description	5
4.2	Code Python : Version sequentielle	5
4.3	Resultat de l'execution	6
5	Version parallele avec Numba	7
5.1	Presentation de Numba	7
5.2	Code Python : Version parallele	7
5.3	Resultat de l'execution	8
6	Calcul du Speedup	8
6.1	Formule : Cours du Prof. TOSSOU (Chapitre 1, diapo 16)	8
6.2	Application numerique	8
6.3	Graphique comparatif	9
6.4	Pourquoi n'atteint-on pas 8 avec 8 threads ?	9
7	Loi d'Amdahl : Proportion parallelisable	9
7.1	Rappel theorique : Cours du Prof. TOSSOU (Chapitre 1, diapo 15) . .	10
7.2	Calcul de la proportion parallelisable P	10
7.3	Resultats	10
7.4	Verification	11
7.5	Speedup maximum theorique	11
7.6	Visualisations	12
8	Analyse et discussion	12
8.1	Resume de l'execution	12
8.2	Ancrage dans les cours du Prof. TOSSOU	13
8.3	Environnement d'execution	14

9 Conclusion	14
References	15

1. Introduction

La programmation parallele est aujourd'hui une competence fondamentale en informatique scientifique et en Big Data. Comme l'enseigne le Professeur TOSSOU dans ses cours, les architectures sequentielles atteignent leurs limites physiques : les processeurs ne peuvent plus augmenter indefiniment leur frequence d'horloge, et les besoins de calcul des applications modernes (simulation scientifique, apprentissage automatique, analyse de donnees massives) exigent des approches paralleles.

Ce travail pratique s'inscrit dans le cadre du cours de Programmation Parallele (Chapitre 1 : Introduction aux machines paralleles, Chapitre 2 : Programmation de machines en memoire partagee). L'objectif est de **paralleliser un calcul de moyenne ponderee sur 1 000 000 d'etudiants** en utilisant la bibliotheque Python **Numba**, puis d'analyser les performances obtenues a travers le **speedup** et la **loi d'Amdahl**.

Pourquoi ce probleme est-il ideal pour la parallelisation ?

Le calcul de moyenne de chaque etudiant est **totalelement independant** des autres : la moyenne de l'etudiant i ne depend en aucune facon de celle de l'etudiant j ($i \neq j$). Il n'existe aucune dependance de donnees entre les iterations. Ce type de probleme est dit *embarrassingly parallel* : chaque tache peut etre distribuee a un coeur different sans synchronisation ni communication entre threads.

Ce paradigme correspond exactement a l'architecture **SIMD** (Single Instruction, Multiple Data) decrite dans le cours du Professeur TOSSOU : une meme instruction (calcul de moyenne) s'applique en parallele sur des flux de donnees differentes (notes d'etudiants differentes).

2. Presentation du probleme

2.1. Enonce et coefficients

L'enonce du TP demande de calculer la moyenne ponderee des notes de $N = 1\,000\,000$ etudiants selon les coefficients suivants :

Table 1. Matieres et coefficients (enonce du TP)

Matiere	Abréviation	Coefficient
Mathematiques	M	5
Physique	P	4
Anglais	A	2
Total		11

2.2. Formule de calcul

La moyenne ponderee de chaque etudiant est calculee selon la formule :

$$\text{Moyenne}_i = \frac{M_i \times 5 + P_i \times 4 + A_i \times 2}{11} \quad (1)$$

3. Generation des donnees

3.1. Approche

Nous generons aleatoirement les notes de 1 000 000 d'étudiants a l'aide de **NumPy**, avec une distribution uniforme entre 0 et 20. Une graine fixe (**seed** = 42) garantit la reproductibilite des resultats. Les donnees sont sauvegardees dans un fichier CSV.

3.2. Code Python : Generation CSV

```
1 import numpy as np
2 import csv, os
3
4 N = 1_000_000
5
6 # Graine fixe pour la reproductibilite
7 np.random.seed(42)
8
9 maths      = np.random.uniform(0, 20, N).astype(np.float64)
10 physique   = np.random.uniform(0, 20, N).astype(np.float64)
11 anglais    = np.random.uniform(0, 20, N).astype(np.float64)
12
13 # Sauvegarde CSV
14 with open("etudiants.csv", "w", newline="") as f:
15     writer = csv.writer(f)
```

```

16     writer.writerow(["id", "maths", "physique", "anglais"])
17     for i in range(N):
18         writer.writerow([i+1, round(maths[i],2),
19                             round(physique[i],2), round(anglais[i]
20                                     ],2)])

```

Listing 1. Generation du fichier CSV avec 1 000 000 d'étudiants

3.3. Resultat de l'exécution

```

[Running] python -u "/Users/NOKHO/Desktop/TP_NUMBA_Programmation_Parallele/tempCodeRunnerFile.py"
ÉTAPE 1 : Génération des données
1,000,000 étudiants générés dans /Users/NOKHO/Desktop/TP_NUMBA_Programmation_Parallele/etudiants.csv
Taille du fichier : 23.0 MB
Formule : (Mx5 + Px4 + Ax2) / 11

```

Figure 1. Etape 1 : Generation et sauvegarde des donnees (VS Code, MacBook M1)

Table 2. Caracteristiques du fichier genere

Parametre	Valeur
Nombre d'étudiants	1 000 000
Taille du fichier CSV	23.0 MB
Distribution des notes	Uniforme [0, 20]
Graine aleatoire	42 (reproductible)

4. Version sequentielle

4.1. Description

La version sequentielle parcourt les N etudiants un par un dans une boucle `for` classique. Pour garantir une comparaison juste avec la version parallele, les **deux versions sont compilees en code machine natif** via le decorateur `@nb.jit(nopython=True)`. Ainsi, la seule difference mesuree est le parallelisme, non l'ecart entre Python interprete et Numba compile.

4.2. Code Python : Version sequentielle

```

1 import numba as nb
2 import time
3

```

```

4 @nb.jit(nopython=True)           # compile en code natif, 1
   thread
5 def calculer_seq(maths, physique, anglais):
6     n = len(maths)
7     moyennes = np.zeros(n, dtype=np.float64)
8     for i in range(n):           # boucle classique,
                                   sequentielle
9         moyennes[i] = (maths[i] * 5 + physique[i] * 4
10                        + anglais[i] * 2) / 11
11     return moyennes
12
13 # Warm-up : compilation JIT sur petit echantillon
14 _ = calculer_seq(maths[:100], physique[:100], anglais[:100])
15
16 # Mesure reelle
17 t_start = time.perf_counter()
18 moyennes_seq = calculer_seq(maths, physique, anglais)
19 T1 = time.perf_counter() - t_start

```

Listing 2. Version sequentielle compilee avec Numba JIT

4.3. Resultat de l'exécution

```

ÉTAPE 2 : Version séquentielle
Temps séquentiel (T1) : 0.0059 secondes
Moyenne générale      : 9.9997 / 20
Min / Max              : 0.1196 / 19.8631

```

Figure 2. Etape 2 : Resultat de la version sequentielle

Table 3. Resultats de la version sequentielle

Indicateur	Valeur
Temps d'exécution T_1	0.0059 s
Moyenne generale	9.9997 / 20
Note minimale	0.1196 / 20
Note maximale	19.8631 / 20
Threads utilises	1

Le temps $T_1 = 0.0059$ s constitue notre **reference** pour le calcul du speedup. La moyenne generale de $9.9997/20$ est coherente avec une distribution uniforme sur $[0, 20]$,

dont l'esperance theorique est exactement 10.

5. Version parallele avec Numba

5.1. Presentation de Numba

Numba est une bibliotheque Python de compilation Just-In-Time (JIT) qui traduit le code Python en code machine natif optimise. Pour activer le parallelisme, il suffit de deux modifications par rapport a la version sequentielle :

- Ajouter `parallel=True` au decorateur `@nb.jit`
- Remplacer `range()` par `nb.prange()` (*parallel range*)

Numba distribue alors automatiquement les iterations entre tous les threads disponibles. Ce modele correspond exactement a la **memoire partagee** du Chapitre 2 du cours : les tableaux NumPy sont accessibles par tous les threads en lecture, et chaque thread ecrit dans sa propre portion du tableau `moyennes` sans conflit.

5.2. Code Python : Version parallele

```
1 @nb.jit(nopython=True, parallel=True)  # parallel=True active
   les threads
2 def calculer_par(maths, physique, anglais):
3     n = len(maths)
4     moyennes = np.zeros(n, dtype=np.float64)
5     for i in nb.prange(n):              # prange distribue
        entre threads
6         moyennes[i] = (maths[i] * 5 + physique[i] * 4
7                        + anglais[i] * 2) / 11
8     return moyennes
9
10 # Warm-up : compilation JIT
11 _ = calculer_par(maths[:100], physique[:100], anglais[:100])
12
13 # Mesure reelle
14 t_start = time.perf_counter()
15 moyennes_par = calculer_par(maths, physique, anglais)
16 Tp = time.perf_counter() - t_start
```

Listing 3. Version parallele avec `parallel=True` et `prange()`

5.3. Resultat de l'exécution

```
ÉTAPE 3 : Version parallèle (Numba)
Compilation JIT en cours...
Nombre de threads      : 8
Temps parallèle (Tp) : 0.0011 secondes
Moyenne générale      : 9.9997 / 20
Les résultats de la moyenne générale sont identiques (séquentiel = parallèle)
```

Figure 3. Etape 3 : Resultat de la version parallele (8 threads, MacBook M1)

Table 4. Resultats de la version parallele

Indicateur	Valeur
Temps d'exécution T_p	0.0011 s
Nombre de threads Numba	8
Moyenne generale	9.9997 / 20
Coherence des resultats	Identiques (sequentiel = parallele)

6. Calcul du Speedup

6.1. Formule : Cours du Prof. TOSSOU (Chapitre 1, diapo 16)

Le speedup est defini dans le cours comme le rapport du temps sequentiel sur le temps parallele :

$$S_p = \frac{T_1}{T_p} \quad (2)$$

ou T_1 est le temps sequentiel et T_p le temps parallele avec p processeurs.

6.2. Application numerique

```
ÉTAPE 4 : Speedup
Formule : S_p = T1 / Tp
T1 = 0.0059 s
Tp = 0.0011 s
N = 8 threads
Speedup S = 5.3006
→ La version parallèle est 5.3× plus rapide
```

Figure 4. Etape 4 : Calcul du speedup

$$S_p = \frac{T_1}{T_p} = \frac{0.0059}{0.0011} = \mathbf{5.3006}$$

Speedup obtenu : $S = 5.30$

La version parallele est **5.3 fois plus rapide** que la version sequentielle sur 8 threads.

6.3. Graphique comparatif

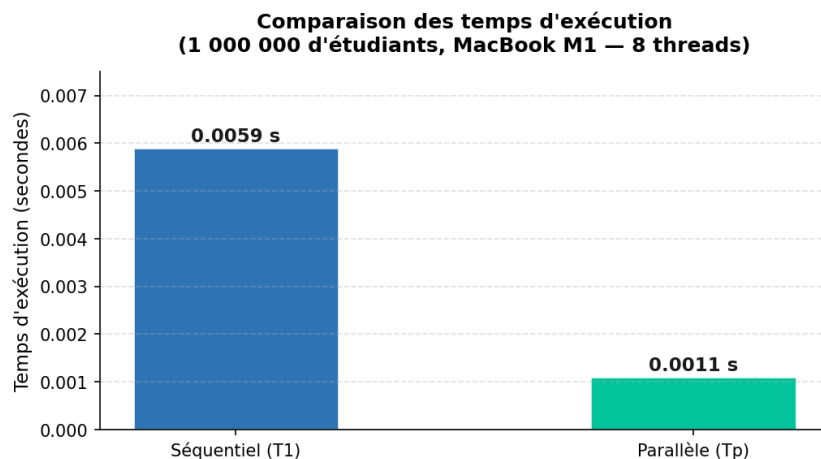


Figure 5. Comparaison des temps d'exécution sequentiel vs parallele

6.4. Pourquoi n'atteint-on pas 8 avec 8 threads ?

Il est naturel de s'attendre a un speedup de 8 avec 8 threads. En realite, plusieurs facteurs l'en empechent :

- **Fraction sequentielle incompressible** : l'initialisation des tableaux, la gestion des threads, les appels systeme ne peuvent pas etre parallelises.
- **Contention memoire** : 8 threads accedent simultanement aux memes tableaux en lecture, ce qui peut saturer le bus memoire.
- **Overhead de distribution** : Numba doit repartir et synchroniser le travail entre les threads, ce qui a un cout non nul.

C'est precisement ce que formalise la **Loi d'Amdahl**.

7. Loi d'Amdahl : Proportion parallelisable

7.1. Rappel theorique : Cours du Prof. TOSSOU (Chapitre 1, diapo 15)

La loi d'Amdahl stipule que si P est la proportion du programme parallélisable et $(1 - P)$ la proportion séquentielle incompressible, alors l'accélération maximale avec N processeurs est :

$$A = \frac{1}{(1 - P) + \frac{P}{N}} \quad (3)$$

7.2. Calcul de la proportion parallélisable P

On dispose du speedup mesure $A = 5.3006$ et du nombre de threads $N = 8$. On inverse algebriquement la formule pour isoler P :

$$A = \frac{1}{(1 - P) + \frac{P}{N}} \implies 1 - P \left(1 - \frac{1}{N}\right) = \frac{1}{A}$$

$$P = \frac{1 - \frac{1}{A}}{1 - \frac{1}{N}} \quad (4)$$

Application numerique :

$$P = \frac{1 - \frac{1}{5.3006}}{1 - \frac{1}{8}} = \frac{1 - 0.1887}{1 - 0.125} = \frac{0.8113}{0.875} = \mathbf{0.9272}$$

7.3. Resultats

```
ÉTAPE 5 : Loi d'Amdahl
Formule : A = 1 / ((1-P) + P/N)
Speedup mesuré (A)      = 5.3006
N (threads)             = 8
Proportion parallélisable = 92.72 %
Proportion séquentielle  = 7.28 %
Speedup max théorique    = 13.75x (N → ∞)

Vérification : A recalculé = 5.3006 ≈ 5.3006
```

Figure 6. Etape 5 : Application de la loi d'Amdahl

Table 5. Resultats de l'analyse par la loi d'Amdahl

Parametre	Valeur
Speedup mesure A	5.3006
Nombre de threads N	8
Proportion parallelisable P	92.72 %
Proportion sequentielle $(1 - P)$	7.28 %
Speedup max theorique $(N \rightarrow \infty)$	13.75

7.4. Verification

On reinjecte $P = 0.9272$ dans la formule d'Amdahl :

$$A_{\text{calc}} = \frac{1}{(1 - 0.9272) + \frac{0.9272}{8}} = \frac{1}{0.0728 + 0.1159} = \frac{1}{0.1887} \approx 5.3006 \quad (\text{OK})$$

7.5. Speedup maximum theorique

Lorsque $N \rightarrow \infty$, le terme $P/N \rightarrow 0$ et la formule devient :

$$A_{\text{max}} = \frac{1}{1 - P} = \frac{1}{1 - 0.9272} = \frac{1}{0.0728} \approx \mathbf{13.75}$$

Meme avec un nombre **infini de processeurs**, ce programme ne pourra jamais dépasser un speedup de **13.75**, a cause des 7.28 % de code sequentiel irreductible. C'est la lecon fondamentale de la Loi d'Amdahl.

7.6. Visualisations

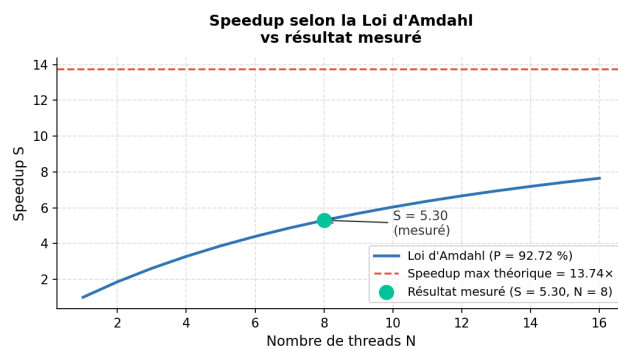


Figure 7. Speedup théorique (Amdahl) vs résultat mesuré

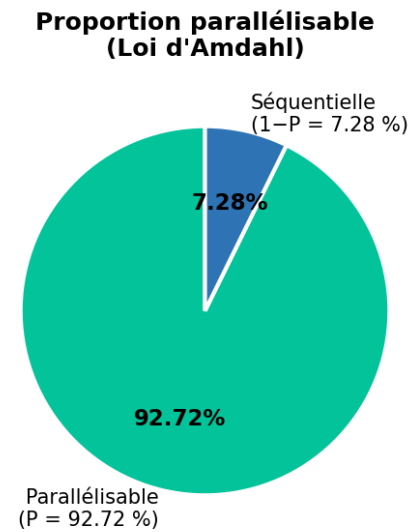


Figure 8. Proportion parallélisable

8. Analyse et discussion

8.1. Resume de l'exécution

RÉSUMÉ DES RÉSULTATS	
Étudiants traités	: 1,000,000
Temps séquentiel (T1)	: 0.0059 s
Temps parallèle (Tp)	: 0.0011 s
Threads Numba (N)	: 8
Speedup (S)	: 5.3006
Proportion parallélisable	: 92.72 %
Proportion séquentielle	: 7.28 %
Speedup max théorique	: 13.75x

Figure 9. Resume final : Recapitulatif de tous les resultats

Table 6. Tableau de synthese complet

Indicateur	Sequentiel	Parallele
Outil	Numba JIT (1 thread)	Numba JIT + prange
Temps d'exécution	0.0059 s	0.0011 s
Speedup	1.0 (reference)	5.30
Coherence resultats	Reference	Identique
Proportion parallelisable	:	92.72 %
Speedup max theorique	:	13.75

8.2. Ancrage dans les cours du Prof. TOSSOU

Architecture SIMD (Ch. 1, diapo 7–9) : Notre calcul correspond au paradigme SIMD : toutes les unites de traitement executent la meme instruction (formule de moyenne) sur des flux de donnees differents (notes d'etudiants differents). Numba exploite les registres vectoriels AVX/NEON du M1.

Memoire partagee (Ch. 2) : Les tableaux NumPy sont accessibles en lecture par tous les threads simultanement. Il n'y a aucun conflit d'ecriture car chaque thread ecrit dans une portion disjointe du tableau de resultats.

Speedup et scalabilite (Ch. 1, diapo 16) : Le speedup $S_p = T_1/T_p = 5.30$ mesure l'acceleration reelle. Nous atteignons 66 % du speedup ideal (8), ce qui est un tres bon resultat pour un calcul en memoire partagee.

Loi d'Amdahl (Ch. 1, diapo 15) : La fraction sequentielle $(1-P) = 7.28\%$ impose une limite absolue de 13.75 independamment du nombre de processeurs.

8.3. Environnement d'exécution

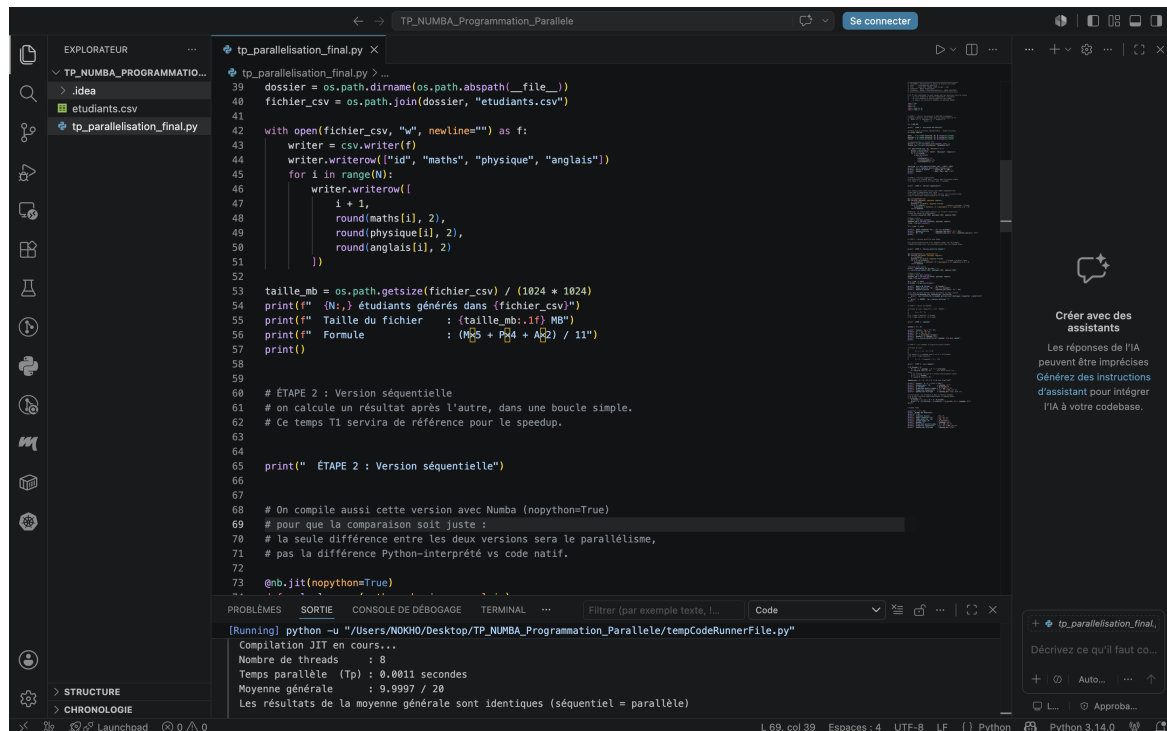


Figure 10. Environnement de developpement : VS Code sur MacBook Pro M1

Table 7. Environnement d'exécution

Composant	Detail
Machine	MacBook Pro M1
Processeur	Apple M1 (8 coeurs : 4 performance + 4 efficacite)
Threads Numba detectes	8
Langage	Python 3.14.0
Editeur	Visual Studio Code
Bibliotheques	NumPy, Numba

9. Conclusion

Ce travail pratique a mis en oeuvre l'ensemble des concepts de programmation parallele enseignés par le Professeur TOSSOU. Nous avons parallelise avec succes le calcul de moyenne ponderee de **1 000 000 d'étudiants** en passant d'une boucle sequentielle `range()` vers une boucle parallele `nb.prange()` compilee avec Numba.

Les resultats obtenus sont coherents et significatifs :

- Un **speedup de 5.30** sur 8 threads, representant 66 % du speedup ideal theorique.

- Une **proportion parallélisable de 92.72 %**, confirmant que le calcul de moyenne est quasi-entièrement parallélisable.
- Un **speedup maximum théorique de 13.75** imposé par la fraction séquentielle irréductible, conformément à la Loi d'Amdahl.
- Une **vérification parfaite** : les résultats séquentiels et parallèles sont mathématiquement identiques.

Ce TP illustre concrètement pourquoi les architectures parallèles sont nécessaires (Chapitre 1) et comment les exploiter efficacement en Python via Numba (Chapitre 2). Il démontre la puissance de la Loi d'Amdahl comme outil d'analyse : même avec un speedup impressionnant, la fraction séquentielle fixe une limite absolue qu'aucun nombre de processeurs ne peut dépasser.

References

1. **TOSSOU, O. N.** : *Cours de Programmation Parallèle, Chapitre 1 : Introduction aux machines parallèles*. AIMS Senegal / Université Virtuelle du Senegal, 2024/2025.
2. **TOSSOU, O. N.** : *Cours de Programmation Parallèle, Chapitre 2 : Programmation de machines en mémoire partagée*. AIMS Senegal / Université Virtuelle du Senegal, 2024/2025.
3. **Documentation officielle Numba** : *Numba : A High Performance Python Compiler*. Disponible sur : <https://numba.readthedocs.io>
4. **SENGHOR, A.** : *Contribution à l'élaboration de compilateurs-paralléliseurs de programmes Java ciblant les architectures à processeurs multi-cœurs*. Référence [1] du cours.
5. **BENHAMIDA, N.** : *Support de cours d'Architectures parallèles*. Référence [2] du cours.
6. **GOUBAULT, E. (Ecole Polytechnique)** : Cours de programmation parallèle. Disponible sur : <https://www.enseignement.polytechnique.fr/profs/informatique/Eric.Goubault/poly/cours008.html>